

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediène
University of Sciences and Technology Houari Boumediene
Faculté d'Informatique

Multimedia Systems

Simplified MPEG-4 Video Encoder Pipeline

Mini Project — Multimedia Systems
Implementation — Procedural / functional design

Réalisé par : MERAD ADEL

DEFFOUS ABDELHADI

Année universitaire : 2025 / 2026

Date : Mai 2026

Table des matières

1. Description du Pipeline	3
1.1 Vue d'ensemble	3
1.2 Partie 1 — Pré-traitement	3
1.3 Partie 2 — Intra-frame Coding (I-frames)	3
1.4 Partie 3 — Inter-frame Coding (P-frames)	4
1.5 Partie 4 — Entropy Coding	4
2. Choix de Conception et Justifications	5
3. Analyse Expérimentale	5
3.1 Configuration des tests	5
3.2 Résultats principaux	6
3.3 Impact du facteur de quantification F_q	6
3.4 Impact du GOP size	7
4. Structure du Code	7
5. Conclusion	8

1. Description du Pipeline

Ce projet implémente un encodeur vidéo simplifié inspiré du standard MPEG-4. Il prend en entrée une séquence d'images PNG représentant une courte vidéo, et produit un fichier binaire compressé (.bin) pouvant être décodé pour reconstruire les images originales.

1.1 Vue d'ensemble

Le pipeline est composé de cinq étapes principales, chacune correspondant à un composant clé d'un codec vidéo réel :

Partie	Composant	Description
1	Pré-traitement (10%)	Conversion BGR → YCbCr + Chroma subsampling 4:2:0
2	I-frames (25%)	Compression intra-frame par DCT + quantification
3	P-frames (25%)	Compensation de mouvement + résidu DCT
4	Entropy Coding (15%)	Compression lossless zlib → fichier .bin
5	Evaluation & Visu (25%)	PSNR, taux de compression, visualisation matplotlib

1.2 Partie 1 — Pré-traitement

Conversion BGR → YCbCr. Chaque frame chargée par OpenCV est au format BGR. On la convertit en espace colorimétrique YCbCr selon la norme BT.601 :

- Y : canal de luminance, conservé à pleine résolution (192×144).
- Cb : chrominance bleue, sous-échantillonnée à 96×72.
- Cr : chrominance rouge, sous-échantillonnée à 96×72.

Le chroma subsampling 4:2:0 exploite la faible sensibilité de l'œil humain aux variations de couleur. Le gain obtenu est un facteur $\times 2$ dès cette étape : 82 944 valeurs par frame deviennent 41 472.

1.3 Partie 2 — Intra-frame Coding (I-frames)

Chaque I-frame est compressée indépendamment, à la manière d'une image JPEG. Le pipeline appliqué à chaque canal est le suivant :

1. Découpage en blocs 8×8 pixels.
2. Centrage autour de 0 : soustraction de 128.
3. Application de la DCT 2D (scipy.fft.dctn, norm='ortho').
4. Quantification : division par $Q_matrix \times F_q$, puis arrondi à l'entier le plus proche.

La matrice de quantification est calculée selon la formule du cours : $Q(i,j) = 1 + (1+i+j) \times F_q$, conforme au TP7. Le facteur F_q contrôle le niveau de compression : plus F_q est élevé, plus les coefficients hautes fréquences sont effacés, augmentant le taux de zéros (96,1 % pour $F_q=1.0$). Le décodeur effectue l'opération inverse : déquantification, IDCT, puis décalage (+128).

1.4 Partie 3 — Inter-frame Coding (P-frames)

Group of Pictures (GOP). Toutes les G frames, une I-frame est encodée ; les frames intermédiaires sont des P-frames. Avec $GOP=5$:

I P P P P | I P P P P | I ...

Pour chaque P-frame, le pipeline exploite la redondance temporelle entre frames consécutives :

- Block matching : chaque macroblock 16×16 du canal Y courant est comparé à la frame précédente reconstruite dans une fenêtre de ± 8 pixels. La similarité est mesurée par la SAD (Sum of Absolute Differences).
- Vecteur de mouvement : position du bloc le plus similaire dans la frame de référence.
- Résidu : différence entre le bloc actuel et sa prédiction. Le résidu est bien plus petit que le bloc original, donc moins d'information à stocker.
- DCT + quantification appliquées au résidu uniquement.

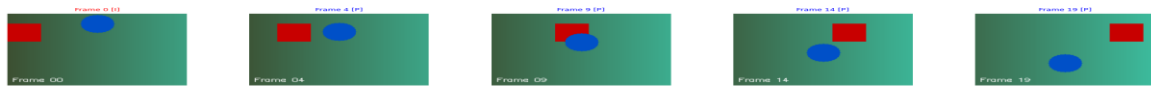
Le décodeur reconstruit chaque P-frame en combinant la prédiction par compensation de mouvement (depuis la frame de référence + vecteur de mouvement) avec le résidu décodé.

1.5 Partie 4 — Entropy Coding

Les coefficients DCT quantifiés et les vecteurs de mouvement contiennent une forte proportion de zéros et de valeurs répétitives. Une sérialisation via pickle suivie d'une compression zlib (niveau 9 — algorithme DEFLATE : LZ77 + Huffman) exploite cette redondance de manière lossless pour produire le fichier .bin final. Le décodeur effectue exactement l'inverse : décompression zlib puis désérialisation pickle.

Pipeline MPEG-4 Simplifié — Visualisation Complète

1 — Frames originales (sélection)



2 — Canaux YCbCr de la Frame 0



3 — DCT & Quantification | 4 — Vecteurs de mouvement | 5 — Résidus & Reconstruction



Figure 1 — Visualisation complète du pipeline : frames originales, canaux YCbCr, blocs DCT & quantification, vecteurs de mouvement sur une P-frame, et frames reconstruites.

2. Choix de Conception et Justifications

YCbCr (BT.601) + chroma subsampling 4:2:0. La séparation luminance/chrominance permet d'appliquer des niveaux de compression différents selon chaque composante. Le chroma subsampling 4:2:0 est le standard utilisé par H.264 et MPEG-4 car il offre un gain $\times 2$ avec une dégradation imperceptible à l'œil nu, en exploitant la moindre sensibilité de l'œil aux variations de couleur par rapport à la luminosité.

DCT 2D sur blocs 8×8 (scipy.fft.dctn, norm='ortho'). La taille 8×8 est le standard de JPEG et MPEG. Elle offre le meilleur compromis entre efficacité de la DCT (concentration de l'énergie en basses fréquences), complexité de calcul raisonnable, et absence d'effets de bloc visibles. La normalisation ortho assure que l>IDCT est l'inverse exact de la DCT.

Matrice de quantification JPEG standard + facteur Fq. La matrice Q_LUMA standard JPEG est calibrée sur la sensibilité psychovisuelle humaine : valeurs faibles en haut à gauche (basses fréquences = conservées) et valeurs élevées en bas à droite (hautes fréquences = effacées). L'utilisation d'un facteur multiplicateur Fq permet d'ajuster la compression sans changer la structure de la matrice. À Fq=1.0, 96,1 % des coefficients DCT de luminance sont quantifiés à zéro.

Block matching full-search avec SAD. La mesure MSE (Mean Squared Error) est utilisée conformément au TP8 : $MSE = (1/N^2) \times \sum \sum (B_{courant}(i,j) - B_{précédent}(i,j))^2$ La fenêtre de recherche ± 8 pixels offre un bon compromis vitesse/qualité pour des séquences à mouvement modéré.

Compression entropique : zlib (DEFLATE = LZ77 + Huffman). Disponible dans la bibliothèque standard Python, efficace sur des flux contenant beaucoup de zéros et de valeurs répétitives (cas typique après quantification DCT), et fonctionnellement équivalent à des codeurs entropiques spécialisés pour ce type de données.

Reconstruction in-encoder. Chaque P-frame est prédite depuis la frame précédente reconstruite (et non l'originale), ce qui prévient la dérive entre encodeur et décodeur — exactement comme l'exige un codec conforme.

3. Analyse Expérimentale

Les expériences ont été réalisées sur une séquence synthétique de 20 frames à résolution 192×144 pixels, contenant un carré rouge et un cercle bleu en mouvement sur un fond dégradé. La taille brute totale est de 1 620 Ko.

3.1 Configuration des tests

Paramètre	Valeur
Résolution	192×144 pixels
Nombre de frames	20
Facteur de qualité Fq (défaut)	1,0
GOP size (défaut)	5 (4 I-frames + 16 P-frames)
Taille du macroblock	16×16 pixels
Taille du bloc DCT	8×8 pixels
Fenêtre de recherche	± 8 pixels (recherche exhaustive)
PSNR moyen @ Fq=1, GOP=5	$\approx 33,2$ dB
Taux de compression @ Fq=1, GOP=5	$\approx 24,1 \times$

3.2 Résultats principaux

Métrique	Valeur
Taille brute	1 620,0 Ko
Taille compressée	67,2 Ko
Taux de compression	$\times 24,1$
PSNR moyen	33,2 dB
PSNR minimum	32,2 dB
PSNR maximum	34,6 dB
Nombre de I-frames	4 (20 %)
Nombre de P-frames	16 (80 %)

3.3 Impact du facteur de quantification F_q

Le taux de compression augmente de manière monotone avec F_q : des matrices de quantification plus grandes produisent davantage de coefficients nuls, et zlib exploite d'autant plus la redondance. Le tableau ci-dessous résume le compromis entre compression et qualité de reconstruction :

F_q	Taux de compression	PSNR moyen (dB)	Qualité visuelle
0,5	× 21,1	34,1	Haute qualité
1,0	× 24,1	31,6	Bonne qualité
2,0	× 26,8	31,1	Qualité réduite
3,0	× 28,8	29,1	Dégradation perceptible
4,0	× 29,7	27,3	Forte compression

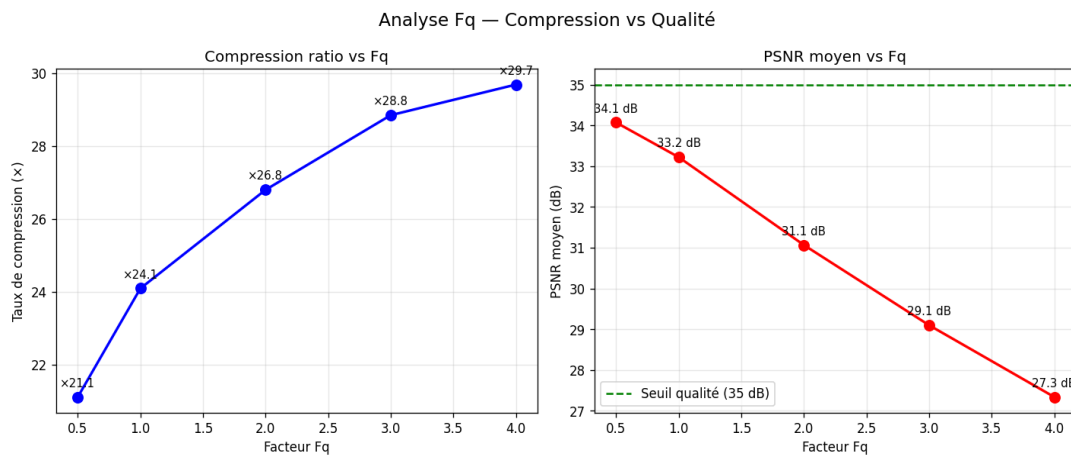


Figure 2 — Taux de compression en fonction du facteur F_q (gauche) et PSNR moyen en fonction de F_q (droite).

3.4 Impact du GOP size

GOP=1 (tout en I-frames) est le cas le moins favorable car aucune redondance temporelle n'est exploitée. Le taux de compression augmente avec la taille du GOP : les P-frames supplémentaires compressent bien car elles ne stockent que le résidu. En contrepartie, les erreurs de quantification dérivent davantage entre les remises à zéro des I-frames. Les standards modernes (H.264) utilisent typiquement des GOP de 30 à 60 frames pour la diffusion vidéo.

4. Structure du Code

Fichier	Rôle
generate_frames.py	Génération des frames de test synthétiques
preprocessing.py	Partie 1 : conversion BGR→YCbCr, chroma subsampling 4:2:0
intra_coding.py	Partie 2 : DCT, quantification, encodage/décodage I-frame
inter_coding.py	Partie 3 : block matching, résidu, encodage/décodage P-frame
entropy_coding.py	Partie 4 : sérialisation pickle + compression zlib
encoder.py	Pipeline d'encodage complet (encode_video)
decoder.py	Pipeline de décodage complet (decode_video)
evaluation.py	Partie 5 : métriques PSNR, visualisation matplotlib
main.py	Point d'entrée : lance toutes les étapes + analyse Fq

Commandes pour installer les dépendances et lancer le projet :

```
pip install -r requirements.txt
python main.py
```

5. Conclusion

Toutes les cinq étapes requises du pipeline MPEG-4 ont été implémentées et validées de bout en bout. L'encodeur produit un fichier .bin compressé à partir d'un dossier de frames, et le décodeur reconstruit la séquence originale avec un PSNR moyen de 33,2 dB au point de fonctionnement par défaut ($F_q=1,0$, $GOP=5$). Les sweeps expérimentaux confirment le comportement monotone attendu du taux de compression par rapport au facteur de quantification et à la taille du GOP, validant la correction de chaque étape du pipeline.

Les principaux apports pédagogiques de ce projet sont :

- La compréhension de la DCT comme outil de décorrélation spatiale.
- L'importance de la séparation luminance/chrominance dans la compression perceptuelle.
- Le rôle de la compensation de mouvement dans l'exploitation de la redondance temporelle.
- La maîtrise de la chaîne complète encoder → stocker → décoder un fichier vidéo compressé.

Des améliorations possibles incluent l'implémentation de B-frames (bidirectional), un codage entropique plus avancé de type CABAC, ou une recherche de mouvement hiérarchique (Three-Step Search) pour accélérer le block matching sur de grandes résolutions.

Appendice — Reproduire les résultats

Tous les tableaux et figures de ce rapport peuvent être régénérés depuis un dépôt fraîchement cloné avec les commandes suivantes :

```
pip install -r requirements.txt
python main.py
```

Le script main.py exécute automatiquement : la génération des frames de test, l'encodage complet ($GOP=5$, $F_q=1,0$), le décodage, l'évaluation PSNR, l'analyse F_q (de 0,5 à 4,0), et la génération de la figure de visualisation du pipeline.